

WS 와 WAS의 차이점

 WS vs WAS &  Spring 핵심 개념 정리

1. WS(Web Server)와 WAS(Web Application Server) 비교

 한눈에 보는 비교표

비교 항목	WS (Web Server)	WAS (Web Application Server)
주요 역할	정적 리소스 제공 및 프록시 역할	동적 비즈니스 로직 처리 및 DB 연동
처리 대상	HTML, CSS, JS, 이미지 등 (이미 완성된 파일)	웹 애플리케이션 수행 결과물 (즉석에서 가공된 데이터)
동작 방식	요청된 URL의 물리적 경로에서 파일을 찾아 그대로 반환	데이터 조회, 연산 등 코드를 실행한 후 결과를 가공해 반환
대표 예시	Nginx, Apache HTTP Server, IIS	Tomcat, JBoss, WebSphere, WildFly

 실무 아키텍처 (WS와 WAS를 분리하여 함께 쓰는 이유)

[클라이언트] → [WS (Nginx)] → (동적 가공 요청) → [WAS (Tomcat)] → [데이터베이스]

- **서버 부하 분산 (Efficiency):** 이미지나 CSS 같은 정적 파일은 가벼운 WS가 앞단에서 직접 처리하고, 비즈니스 로직이 필요한 요청만 WAS로 토스하여 WAS의 자원 낭비를 막습니다.
- **보안 강화 (Security):** WAS에는 실제 회원 정보나 DB 비밀번호 같은 민감한 자산이 연결되어 있습니다. WS를 리버스 프록시(Reverse Proxy)로 앞단에 내세우면 외부에 WAS의 실제 IP 주소를 숨길 수 있습니다.
- **무중단 배포 및 가용성 (High Availability):** 앞단의 WS가 요청을 대기시키거나 여러 대의 WAS로 요청을 나누는 로드 밸런싱(Load Balancing)을 수행하여, 서버가 꺼지지 않고 배포되도록 돕습니다.

Spring 프레임워크 4대 특징

 특징 요약표

특징	개념 및 정의	핵심 목적 / 도입 효과
POJO	특정 프레임워크나 규약에 종속되지 않는 순수한 자바 객체	환경에 영향받지 않는 객체지향 설계, 자유로운 테스트 및 재사용
IoC	객체의 생성, 관리, 생명주기 제어권이 개발자에서 컨테이너로 역전됨	애플리케이션 컴포넌트 간의 느슨한 결합(Loose Coupling) 형성
DI	IoC를 구현하는 기법으로, 외부에서 인터페이스에 구현 객체를 주입함	의존하는 객체가 바뀌어도 내부 코드를 수정할 필요가 없음
AOP	핵심 비즈니스 로직과 공통 부가 기능(로깅, 트랜잭션 등)을 분리	중복 코드를 완벽히 제거하여 핵심 로직의 가독성 향상

구체적인 특징 이해하기

◆ POJO (Plain Old Java Object)

- ✗ POJO가 아닌 코드 (과거 EJB 방식처럼 기술에 종속적인 구조)

java

```
// 특정 프레임워크의 규약을 구현해야만 작동하므로 단독 테스트나
// 재사용이 불가능함
public class LegacyUserService implements javax.ejb.SessionBean {
    public void ejbActivate() {}
    public void ejbPassivate() {}
    public void ejbRemove() {}
    public void setSessionContext(SessionContext ctx) {}

    public void createUser(String id) { /* 비즈니스 로직 */ }
}
```

코드를 사용할 때는 주의가 필요합니다.

- POJO 코드 (Spring 방식)

java

```
// 오직 자바 표준 문법으로만 이루어져 깔끔하며, 프레임워크가 바뀌어도
// 코드 재사용 가능
public class UserService {
    public void createUser(String id) {
```

```

        // 비즈니스 로직에만 집중
    }
}

```

코드를 사용할 때는 주의가 필요합니다.

- ◆ **IoC (제어의 역전) & DI (의존성 주입)**

- **✗ 기존 방식 (개발자가 직접 객체를 만들고 제어함)**

java

```

public class OrderService {
    // OrderService가 직접 할인 정책을 결정하고 생성함
    // 할인 정책을 바꾸려면 이 클래스의 소스 코드를 직접 고쳐야
    하므로 결합도가 높음
    private DiscountPolicy discountPolicy = new FixDiscountPolicy();
}

```

코드를 사용할 때는 주의가 필요합니다.

- **◯ Spring 방식 (IoC / DI 적용)**

java

```

@Service
public class OrderService {
    private final DiscountPolicy discountPolicy;

    // OrderService는 인터페이스만 바라볼 뿐, 어떤 구현체가 들어올지 모름
    // Spring 컨테이너가 애플리케이션 실행 시점에 구현체(예: RateDiscountPolicy)를 생성해서 넣어줌
    @Autowired
    public OrderService(DiscountPolicy discountPolicy) {
        this.discountPolicy = discountPolicy;
    }
}

```

코드를 사용할 때는 주의가 필요합니다.

- ◆ AOP (관점 지향 프로그래밍)

- ✗ AOP가 없는 코드 (핵심 로직과 부가 로직이 사방에 중복되어 혼재함)

java

```
public void registerMember() {
    long startTime = System.currentTimeMillis(); // ◀ 부가 기능: 시간 측정 시작

    System.out.println("회원 가입 처리 중..."); // ◀ 핵심 비즈니스 로직

    long endTime = System.currentTimeMillis(); // ◀ 부가 기능: 시간 측정 종료
    System.out.println("소요 시간: " + (endTime - startTime) + "ms");
}
```

코드를 사용할 때는 주의가 필요합니다.

- ○ AOP가 적용된 코드 (관점의 분리)

java

```
// 1. 부가 기능만 담당하는 공통 Aspect 클래스를 따로 정의
@Aspect
@Component
public class PerformanceAspect {
    @Around("execution(* com.example.service..*(..))")
    // service 패키지 밑의 모든 메서드에 자동 적용
    public Object logExecutionTime(ProceedingJoinPoint joinPoint) throws Throwable {
        long start = System.currentTimeMillis();
        Object result = joinPoint.proceed(); // 원래 실행하려던 핵심 비즈니스 로직 수행
        long end = System.currentTimeMillis();
        System.out.println("소요 시간: " + (end - start) + "ms");
        return result;
    }
}
```

```
}

// 2. 실제 비즈니스 클래스는 원래의 목적에만 집중
@Service
public class MemberService {
    public void registerMember() {
        // 시간 측정 코드가 완벽히 제거됨. 오직 핵심 회원가입
        로직만 작성
        System.out.println("회원 가입 처리 중...");
    }
}
```